

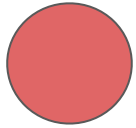
Объектно-ориентированное программирование

Никита Остроухов

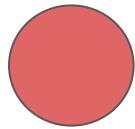
Кафедра информатики МФТИ

Задача движения

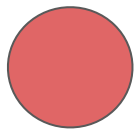
Движение тела



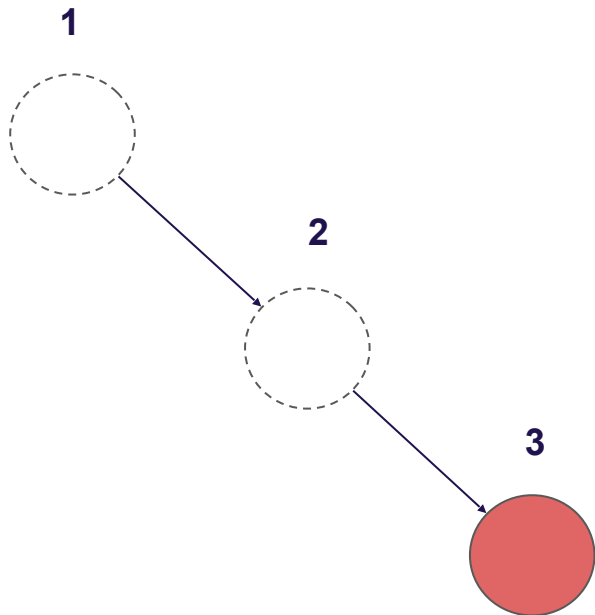
Движение тела



Движение тела

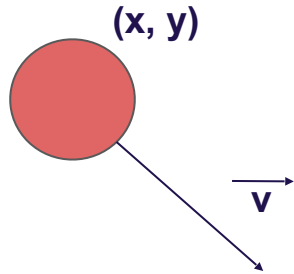


Движение тела



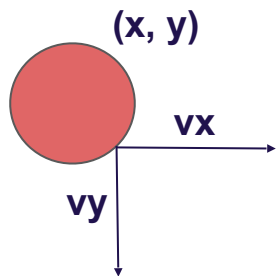
Рассмотрим задачу движения с точки зрения физики

Движение тела



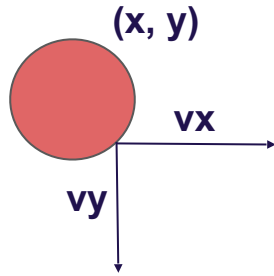
В любой момент времени тело
обладает координатами (x, y)
и вектором скорости $\vec{v}$

Движение тела



Скорость состоит из компонент v_x и v_y

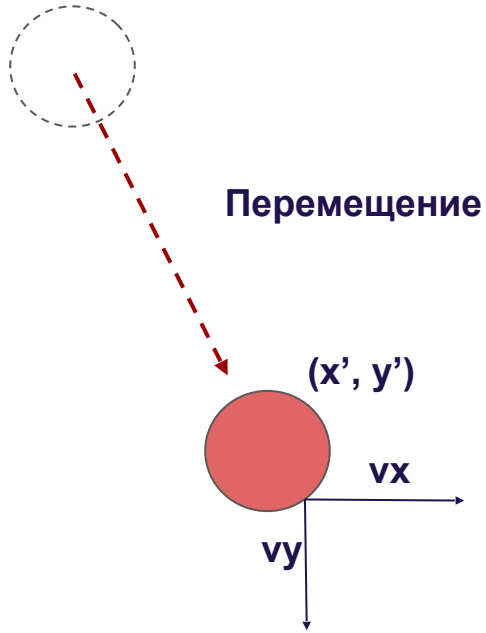
Движение тела



Положение
через dt ?

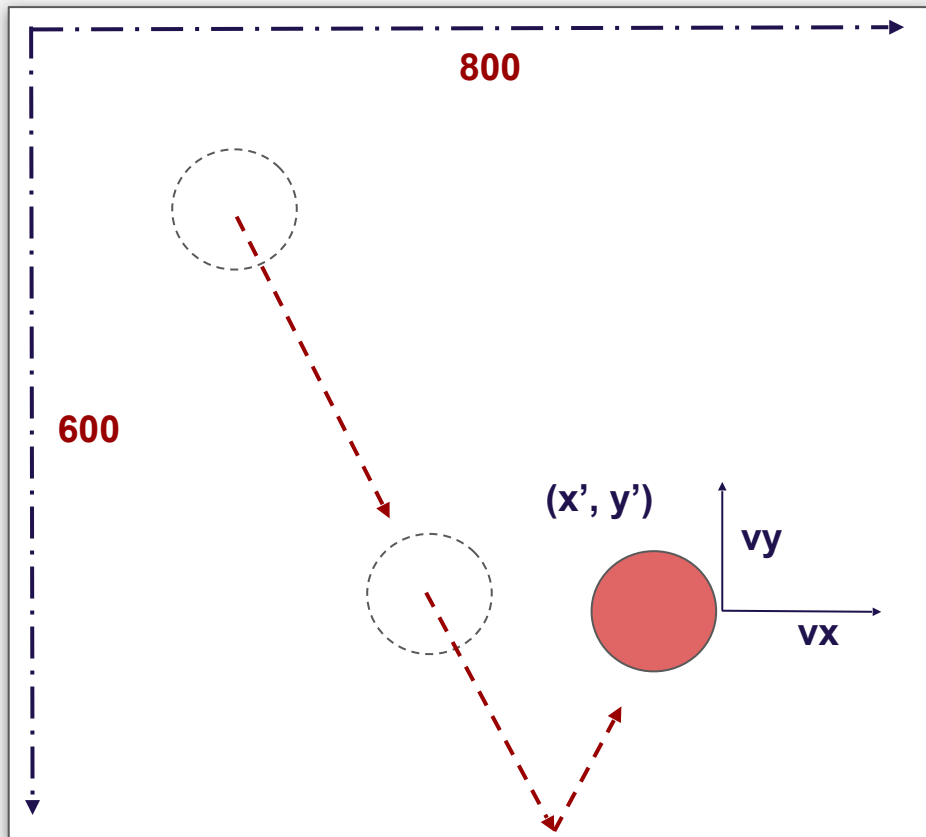
Какие координаты будут у шара через
время dt ?

Движение тела



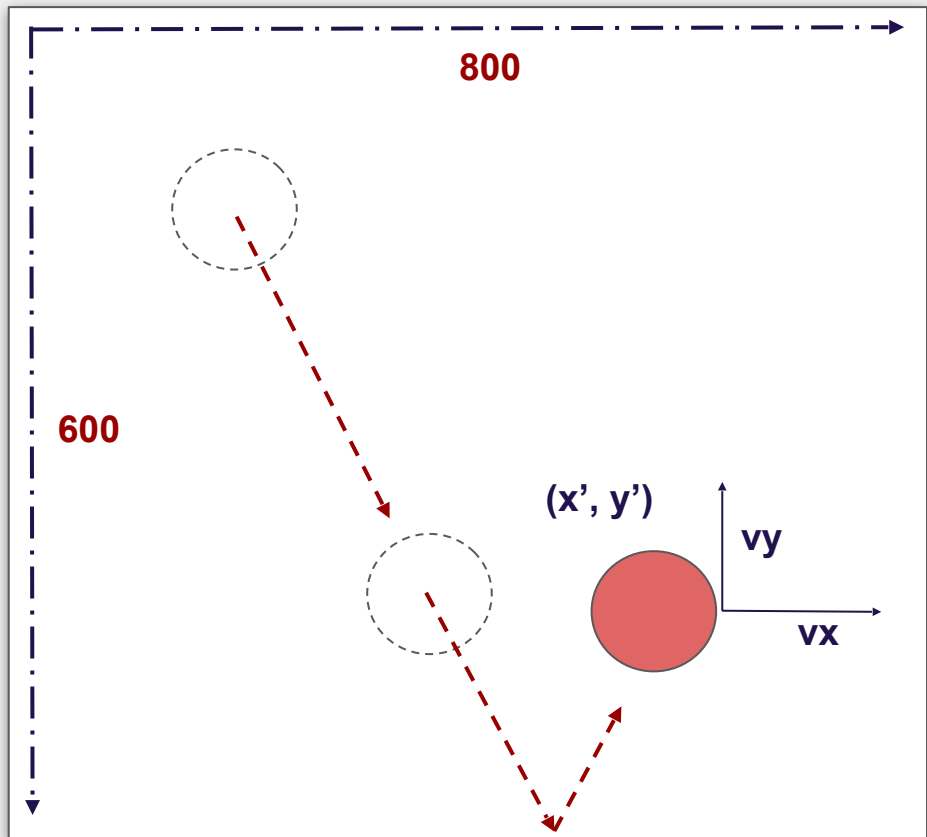
$$x = x + v_x * dt$$
$$y = y + v_y * dt$$

Столкновение со стенками



Какое условие шара отражения от стенок?

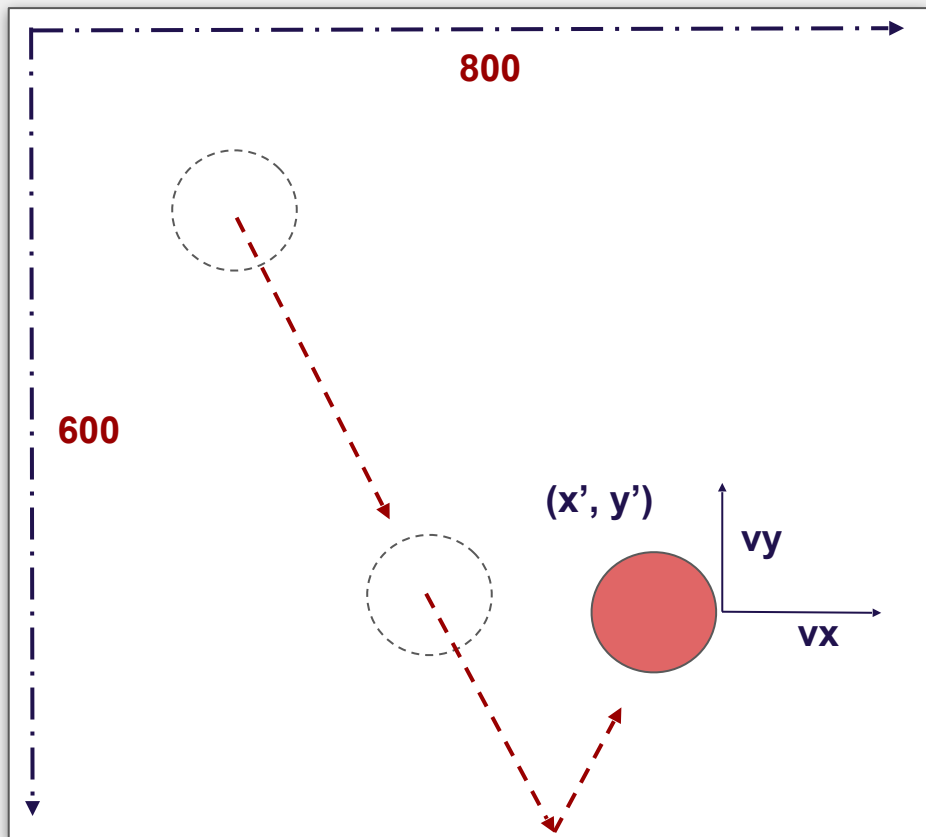
Столкновение со стенками



Условие отражения от **нижней**
стенки

if $y > 600$ $v_y = -v_y$

Столкновение со стенками



Условие отражения от **нижней** стенки

$$\text{if } y > 600 \text{ } v_y = -v_y$$

Условие отражения от **верхней** стенки

$$\text{if } y < 0 \text{ } v_y = -v_y$$

Модель движения

```
dt = 0.1
```

```
x = 400
```

```
y = 300
```

```
vx = 5
```

```
vy = 10
```

```
x = x + vx * dt
```

```
y = y + vy * dt
```

```
if y < 0 or y > 60
```

```
    vy = -vy
```

```
if x < 0 or x > 800
```

```
    vx = -vx
```

```
рисует шар в (x, y)
```

Модель движения

```
dt = 0.1  
x = 400  
y = 300  
vx = 5  
vy = 10
```

```
x = x + vx * dt  
y = y + vy * dt
```

```
if y < 0 or y > 60  
    vy = -vy  
if x < 0 or x > 800  
    vx = -vx
```

рисует шар в (x, y)

```
int main()  
{  
    const float dt = 0.1;  
    int x = 400;  
    int y = 300;  
    int vx = 5;  
    int vy = 10;  
  
    x = x + vx * dt;  
    y = y + vy * dt;  
  
    if (y < 0 || y > 600)  
        vy = -vy;  
    if (x < 0 || x > 800)  
        vx = -vx;  
  
    drawBall(x, y);  
}
```

Бесконечный цикл

```
int main()
{
    ...

    while (true)
    {
        x = x + vx * dt;
        y = y + vy * dt;

        if (y < 0 || y > 600)
            vy = -vy;
        if (x < 0 || x > 800)
            vx = -vx;

        drawBall(x, y);
    }
    ...
}
```

Делаем бесконечный цикл программы

Бесконечный цикл

```
int main()
{
    ...

    for (;;)
    {
        x = x + vx * dt;
        y = y + vy * dt;

        if (y < 0 || y > 600)
            vy = -vy;
        if (x < 0 || x > 800)
            vx = -vx;

        drawBall(x, y);
    }
    ...
}
```

То же самое но более модно

**Задача решится сама собой
если начать с чертежа и модели**

Задание

Добавить ручное управление клавишами

Основной цикл

```
x = x + vx * dt;  
y = y + vy * dt;  
  
if (y < 0 || y > 600)  
    vy = -vy;  
if (x < 0 || x > 800)  
    vx = -vx;  
  
if (GetAsyncKeyState(VK_UP))  
    vy--;  
if (GetAsyncKeyState(VK_DOWN))  
    vy++;  
if (GetAsyncKeyState(VK_LEFT))  
    vx--;  
if (GetAsyncKeyState(VK_RIGHT))  
    vx++;  
  
drawBall(x, y);
```

Разделим код на самостоятельные функции

Основной цикл

```
x = x + vx * dt;  
y = y + vy * dt;
```

```
if (y < 0 || y > 600)  
    vy = -vy;  
if (x < 0 || x > 800)  
    vx = -vx;
```

```
if (GetAsyncKeyState(VK_UP))  
    vy--;  
if (GetAsyncKeyState(VK_DOWN))  
    vy++;  
if (GetAsyncKeyState(VK_LEFT))  
    vx--;  
if (GetAsyncKeyState(VK_RIGHT))  
    vx++;
```

```
drawBall(x, y);
```

движение

управление

отрисовка

Основной цикл

```
x = x + vx * dt;  
y = y + vy * dt;
```

```
if (y < 0 || y > 600)  
    vy = -vy;  
if (x < 0 || x > 800)  
    vx = -vx;
```

```
if (GetAsyncKeyState(VK_UP))  
    vy--;  
if (GetAsyncKeyState(VK_DOWN))  
    vy++;  
if (GetAsyncKeyState(VK_LEFT))  
    vx--;  
if (GetAsyncKeyState(VK_RIGHT))  
    vx++;
```

```
drawBall(x, y);
```

moveBall()

controlBall()

drawBall()

Функция движения

```
void moveBall(int x, int y, int vx, int vy, float dt)
{

}

int main()
{
    ...

    moveBall(x, y, vx, vy, dt)

    ...
}
```

Поступим также как и с функцией
drawBall

Функция движения

```
void moveBall(int x, int y, int vx, int vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;

    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}

int main()
{
    ...

    moveBall(x, y, vx, vy, dt)

    ...
}
```

**Перенесем код движения из main в
moveBall**

Шары перестали двигаться

Почему??

Причина - копирование

Проблема копирования

```
void moveBall(int x, int y, int vx, int vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;

    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}

int main()
{
    int x = 50;

    moveBall(x, y, vx, vy, dt)
    ...
}
```

**Аргумент функции
moveBall**

**Локальная переменная
main**

Это две **разных** переменных

Проблема копирования

```
void moveBall(int x, int y, int vx, int vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;

    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}
```

```
int main()
{
    ...

    moveBall(x, y, vx, vy, dt)

    ...
}
```

**Копирование значений
переменных**



При вызове функции **moveBall()** мы
копируем значения x, y, vx, vy, dt
в аргументы функции

Проблема копирования

```
void moveBall(int x, int y, int vx, int vy, float dt)
```

```
{
```

```
    x = x + vx * dt;
```

```
    y = y + vy * dt;
```

← **Изменение копии**

```
    if (y < 0 || y > 600)
```

```
        vy = -vy;
```

```
    if (x < 0 || x > 800)
```

```
        vx = -vx;
```

```
}
```

```
int main()
```

```
{
```

```
    ...
```

```
    moveBall(x, y, vx, vy, dt)
```

```
    ...
```

```
}
```

**Копирование значений
переменных**

←

Изменение копии **никак не влияет** на
переменные объявленные в **main**

Проблема копирования

```
void moveBall(int x, int y, int vx, int vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;
    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}

int main()
{
    ...
    moveBall(x, y, vx, vy, dt)
    ...
}
```

Изменение копии

Копирование значений переменных

Изменение копии **никак не влияет** на переменные объявленные в **main**



Координаты шара **не обновляются** и он не двигается

Оперативная память



```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

Рассмотрим следующий код

Оперативная память



```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;
    set(x)
}
```

Создание локальной переменной
функции main

Оперативная память



0xFF32E6A6



Адрес этой переменной в памяти

```
void set(int x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;
    set(x);
}
```

Создание локальной переменной
функции main

Оперативная память



↑
0xFF32E6A6

```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

Переменная аргумента функции также
имеет место в памяти

Оперативная память



↑
0xFF32E6A6

↑
0xFF32E6BD

Адрес аргумента функции set

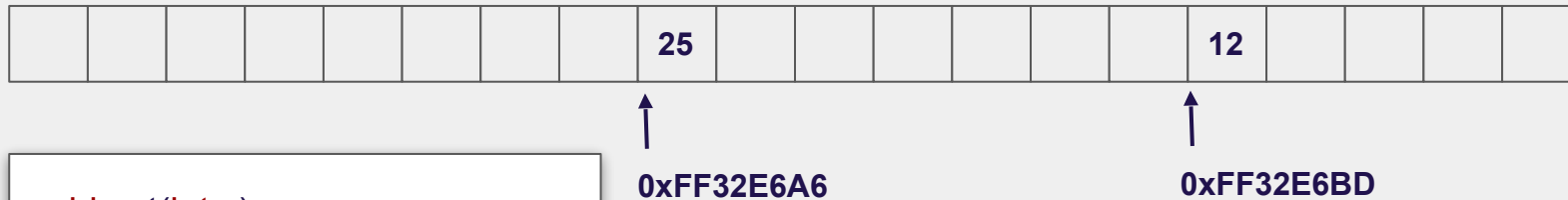
Переменная аргумента функции также
имеет место в памяти

```
void set(int x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;

    set(x)
}
```

Оперативная память



```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

Передаем **значение** переменной в функцию **set**

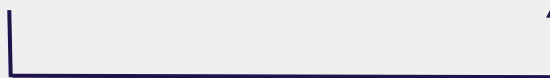
Оперативная память



0xFF32E6A6



0xFF32E6BD



Копирование

```
void set(int x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;

    set(x)
}
```

Передаем **значение** переменной в
функцию **set**

Оперативная память



↑
0xFF32E6A6

↑
0xFF32E6BD

Копия

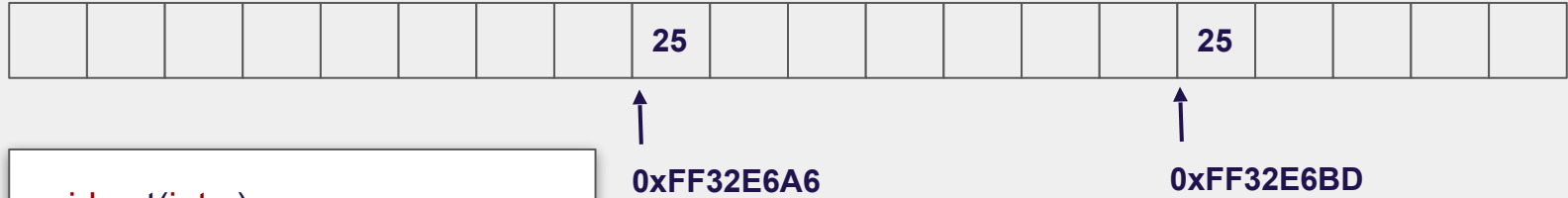
Копирование

```
void set(int x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;
    set(x);
}
```

Передаем **значение** переменной в
функцию **set**

Оперативная память



```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

Присваивание нового значения

Оперативная память



↑
0xFF32E6A6

↑
0xFF32E6BD

```
void set(int x)
{
    x = 15;
}
```

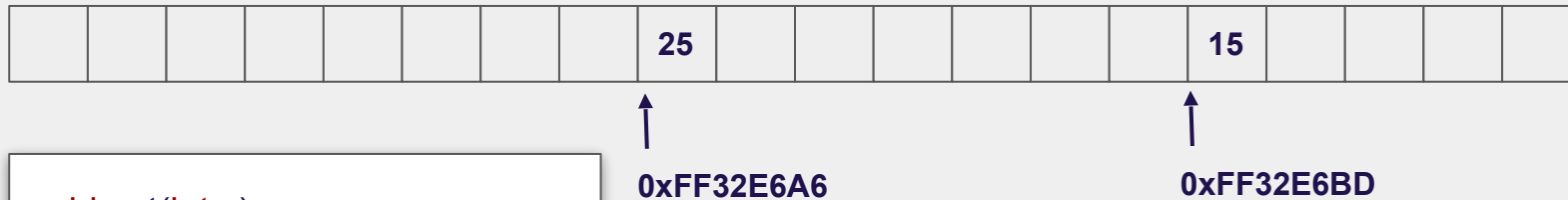
```
int main()
{
    int x = 25;

    set(x)
}
```

**Изменяется аргумент а не локальная
переменная в main**

Присваивание нового значения

Оперативная память



```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

После вызова функции set()
переменная x **не изменилась**

Как избежать копирования?

**Надо передавать не значение переменной
а адрес в памяти где она лежит**

Оперативная память



↑
0xFF32E6A6

```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(x)
}
```

Рассмотрим пример еще раз

Оперативная память



↑
0xFF32E6A6

```
void set(int x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;

    set(x)
}
```

Нужно передать в функцию set **адрес 0xFF32E6A6**
а не значение 25

Оперативная память



↑
0xFF32E6A6

```
void set(int x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(&x)
}
```

Чтобы передать **адрес** а не значение
нужно написать **оператор &**

Оперативная память



↑
0xFF32E6A6

```
void set(int* x)
{
    x = 15;
}

int main()
{
    int x = 25;

    set(&x)
}
```

И указать правильный тип аргумента в функции set

Оперативная память



↑
0xFF32E6A6

```
void set(int* x)
{
    x = 15;
}
```

```
int main()
{
    int x = 25;

    set(&x)
}
```

И указать правильный тип аргумента в функции set

int* - тип **адрес** по которому лежит
переменная типа **int**

Оперативная память



↑
0xFF32E6A6

```
void set(int* x)
```

```
{
```

```
    *x = 15;
```

```
}
```

```
int main()
```

```
{
```

```
    int x = 25;
```

```
    set(&x)
```

```
}
```

Чтобы **присвоить значение** переменной лежащей по этому **адресу** нужно использовать **оператор ***

Оперативная память



↑
0xFF32E6A6

```
void set(int* x)
{
    *x = 15;
}
```

```
int main()
{
    int x = 25;

    set(&x)
}
```

Новое значение **присваивается** по тому **адресу**
который мы **передали** в аргумент функции

Оперативная память



↑
0xFF32E6A6

```
void set(int* x)
{
    *x = 15;
}

int main()
{
    int x = 25;

    set(&x)
}
```

После такого вызова
значение переменной x **поменялось**



Копирование не позволяет менять значение переданной переменной

Передача адреса

```
void moveBall(int* x, int* y, int* vx, int* vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;

    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}

int main()
{
    ...

    moveBall(x, y, vx, vy, dt)

    ...
}
```



Функция moveBall должна принимать
адреса

Передача адреса

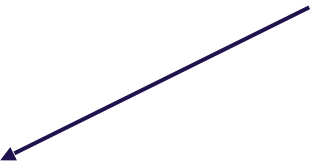
```
void moveBall(int* x, int* y, int* vx, int* vy, float dt)
{
    x = x + vx * dt;
    y = y + vy * dt;

    if (y < 0 || y > 600)
        vy = -vy;
    if (x < 0 || x > 800)
        vx = -vx;
}

int main()
{
    ...

    moveBall(&x, &y, &vx, &vy, dt)

    ...
}
```



Мы должны передавать адреса а не значения

Передача адреса

```
void moveBall(int* x, int* y, int* vx, int* vy, float dt)
{
    *x = *x + *vx * dt;
    *y = *y + *vy * dt;

    if (*y < 0 || *y > 600)
        *vy = -*vy;
    if (*x < 0 || *x > 800)
        *vx = -*vx;
}

int main()
{
    ...

    moveBall(&x, &y, &vx, &vy, dt)

    ...
}
```

Мы должны явно указать что
обращаемся к переменной лежащей по
этому адресу

Задание

Перенести код управления в функцию `controlBall()`